

A AURORA ENTREGA O COMANDO À
INTELIGÊNCIA ARTIFICIAL

INTELIGÊNCIA ARTIFICIAL NO COMANDO

1

LISTA DE FIGURAS

Figura 1 - Núcleo cognitivo da Aurora Siger.....6

Figura 2 - Orientações do núcleo cognitivo da Aurora Siger8



LISTA DE TABELAS

Tabela 1 – Critérios de avaliação19

EMENDAS

SUMÁRIO

INTELIGÊNCIA ARTIFICIAL NO COMANDO	5
1 INTRODUÇÃO – QUANDO OS DADOS PRECISAM VIRAR DECISÃO	5
2 CONTEXTO NARRATIVO – O NASCIMENTO DO NÚCLEO COGNITIVO.....	7
3 CONEXÃO COM AS DISCIPLINAS ATIVADAS NA FASE 5	8
4 OBJETIVOS DE APRENDIZAGEM.....	10
5 ATIVIDADE INTEGRADORA – NÚCLEO COGNITIVO DA AURORA SIGER (NCAS).....	10
5.1 Organização dos dados da colônia	11
5.2 Manipulação de arquivos em Python	12
5.3 Estrutura JSON do sistema	12
5.4 Regras lógicas e simplificação booleana	13
5.5 Engenharia de prompts e simulação de IA generativa	14
5.6 Memória, armazenamento e fluxo de dados	14
5.7 Diversidade, ética e responsabilidade no uso da IA.....	15
6 ENTREGÁVEIS.....	15
6.1 Requisitos do sistema desenvolvido	16
6.2 Arquivos obrigatórios na entrega (.zip).....	17
6.3 Regras técnicas.....	18
7 CRITÉRIOS DE AVALIAÇÃO.....	18
GLOSSÁRIO	20

INTELIGÊNCIA ARTIFICIAL NO COMANDO

1 INTRODUÇÃO – QUANDO OS DADOS PRECISAM VIRAR DECISÃO

Nas fases anteriores, a missão Aurora Siger evoluiu de uma operação de pouso para uma colônia marciana com infraestrutura inteligente e interconectada. Os módulos passaram a trocar informações, compartilhar energia, registrar eventos e depender cada vez mais de sistemas computacionais para manter a estabilidade da base.

Com o avanço da missão, a quantidade de dados gerados aumentou rapidamente. A rede da colônia passou a receber diferentes tipos de informações, como:

- Alertas operacionais enviados pelos módulos;
- Logs de funcionamento dos sistemas;
- Registros de consumo energético;
- Solicitações da tripulação;
- Relatórios técnicos;
- Mensagens entre setores da base.

O problema é que dados armazenados sem organização não geram conhecimento. Quando os registros ficam espalhados, mal estruturados ou difíceis de consultar, a equipe pode enfrentar dificuldades para:

- Localizar informações importantes;
- Interpretar rapidamente mensagens críticas;
- Validar condições de operação;
- Responder a falhas com agilidade;
- Transformar dados em apoio real para a decisão.

INTELIGÊNCIA ARTIFICIAL NO COMANDO

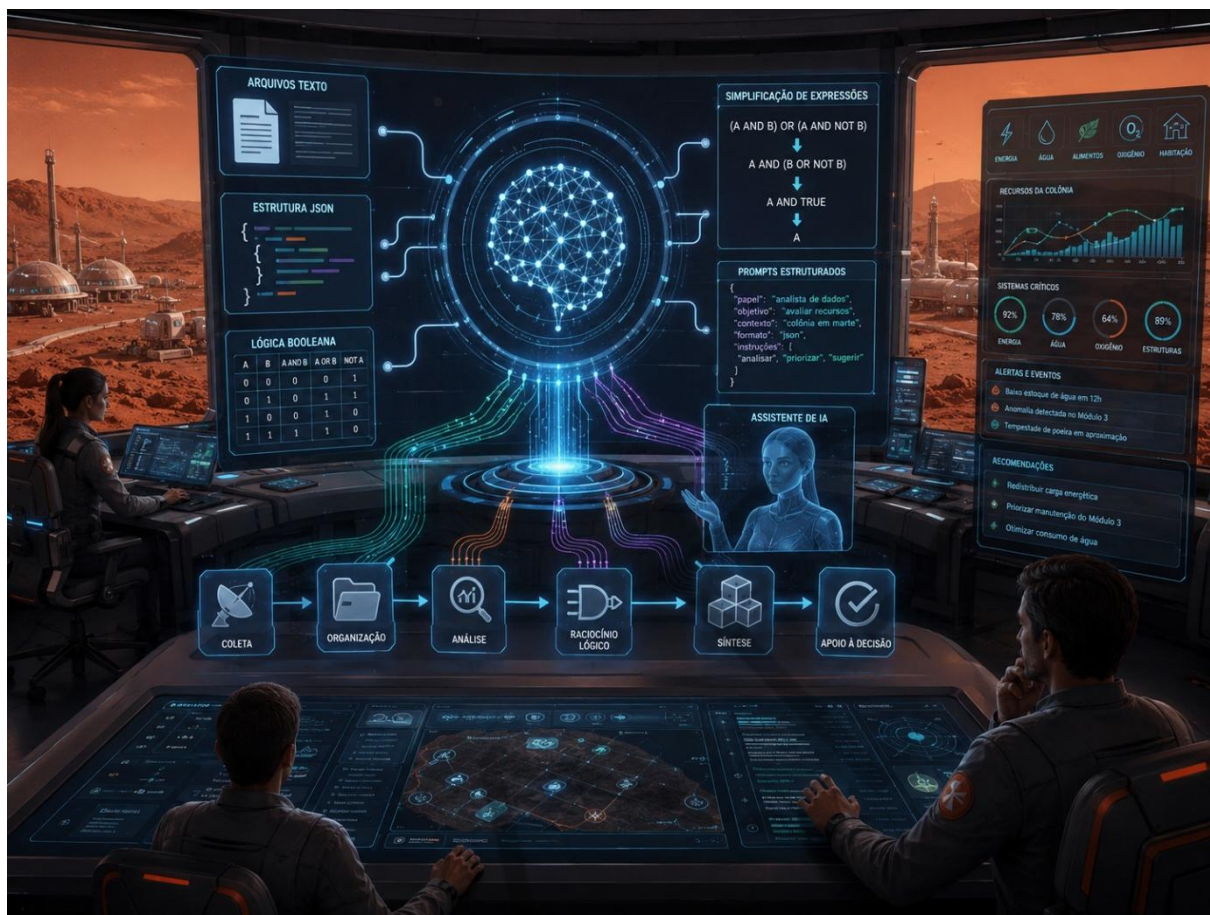


Figura 1 - Núcleo cognitivo da Aurora Siger
Fonte: Elaborada pelo autor (2026)

Diante desse cenário, a Aurora Siger precisa de um novo tipo de sistema: um núcleo cognitivo capaz de organizar informações da colônia, recuperar registros salvos, aplicar regras lógicas e simular interações inteligentes com base em prompts estruturados.

Esse sistema não substituirá os especialistas humanos; ele funcionará como uma ferramenta de apoio para consulta, organização e análise das informações da base, utilizando apenas os recursos estudados nesta fase, como arquivos texto, JSON, lógica booleana, simplificação de expressões, LLMs e engenharia de prompts.

Ao longo dessa fase, você irá desenvolver um protótipo do núcleo cognitivo da Aurora Siger, conectando armazenamento de dados, lógica computacional e simulação de IA generativa em uma solução simples, organizada e coerente com o contexto da missão.

2 CONTEXTO NARRATIVO – O NASCIMENTO DO NÚCLEO COGNITIVO

Após a expansão da infraestrutura da Aurora Siger, a colônia passa a operar com diversos módulos conectados. Cada setor envia informações constantes para o centro de controle: consumo energético, status de equipamentos, registros de manutenção, solicitações da tripulação, alertas de segurança e relatórios de operação.

Inicialmente, esses dados eram analisados manualmente pela equipe técnica. Porém, com o crescimento da colônia, essa abordagem se tornou insuficiente. A equipe começou a enfrentar dificuldades para localizar registros antigos, interpretar rapidamente mensagens críticas e organizar relatórios operacionais.

Além disso, diferentes setores da base passaram a registrar informações em formatos variados. Alguns dados estavam em arquivos texto, outros em estruturas de dados internas e outros precisavam ser organizados em formatos mais padronizados para facilitar a leitura por sistemas computacionais.

Diante desse cenário, a missão define uma nova prioridade: criar o núcleo cognitivo da Aurora Siger, que deverá funcionar como um sistema de apoio computacional capaz de:

- Armazenar registros importantes da colônia;
- Recuperar informações salvas em arquivos;
- Organizar dados em formato JSON;
- Aplicar validações lógicas em solicitações operacionais;
- Simplificar regras booleanas utilizadas no sistema;
- Estruturar prompts para simular interações com IA generativa;
- Apoiar a geração de respostas organizadas e padronizadas;
- Considerar aspectos éticos, sociais e de diversidade no uso da tecnologia.

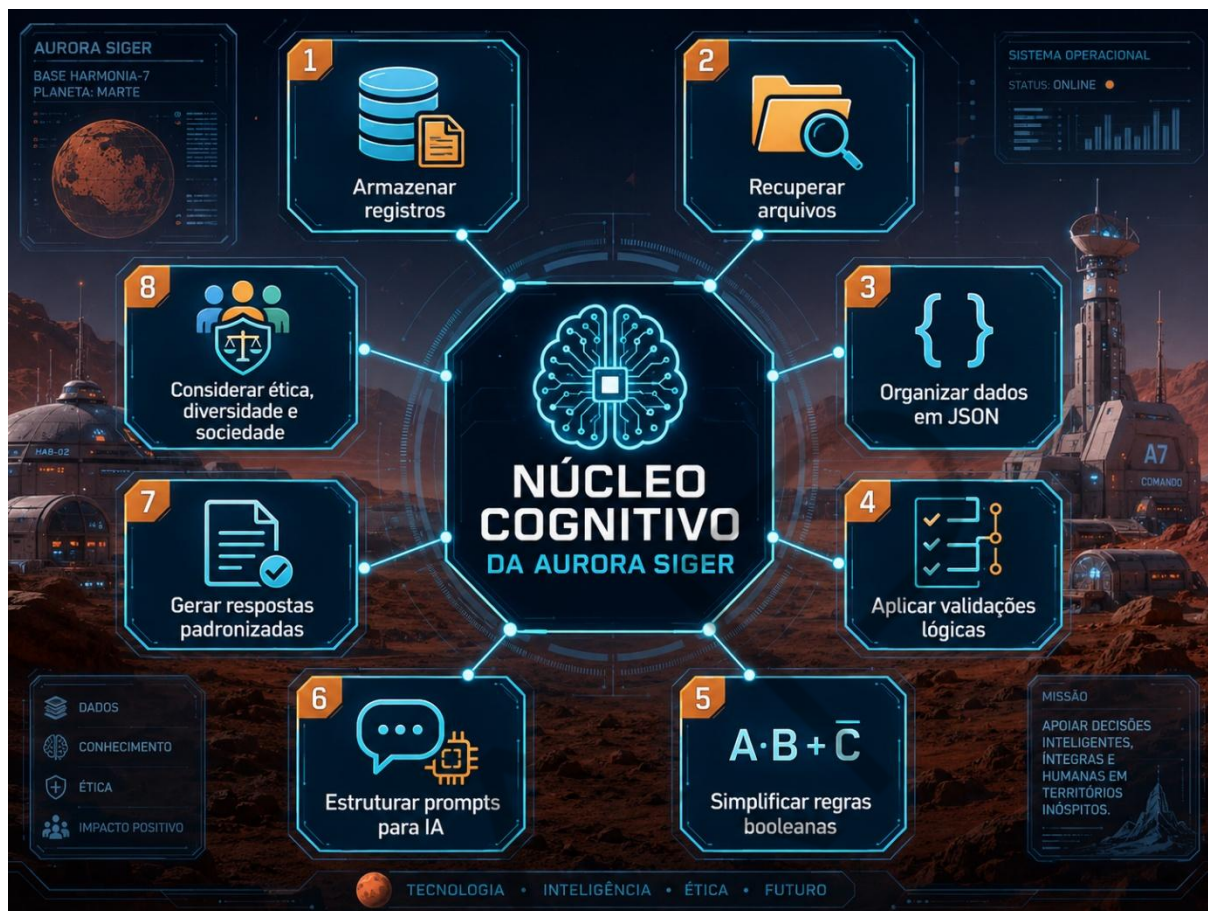


Figura 2 - Orientações do núcleo cognitivo da Aurora Siger
Fonte: Elaborada pelo autor (2026)

O núcleo cognitivo representa a transição da Aurora Siger de uma infraestrutura inteligente para uma colônia com capacidade de apoio computacional à decisão.

Aqui, você não deverá treinar modelos de IA, criar redes neurais ou desenvolver sistemas avançados fora do conteúdo estudado; o foco será construir um protótipo coerente, utilizando os recursos vistos nos materiais.

3 CONEXÃO COM AS DISCIPLINAS ATIVADAS NA FASE 5

O desenvolvimento do núcleo cognitivo da Aurora Siger depende diretamente da integração dos conteúdos estudados nesta fase do curso.

Na disciplina de pensamento computacional e automação com Python, você utilizará os conceitos de manipulação de arquivos texto e JSON para gravar, recuperar, editar e organizar informações da colônia. Esses conteúdos serão importantes para que o sistema consiga manter registros persistentes, como logs, solicitações, respostas e dados operacionais.

INTELIGÊNCIA ARTIFICIAL NO COMANDO

Na disciplina de *Computer Science*, você aplicará os teoremas de simplificação e os de De Morgan para representar e simplificar regras lógicas utilizadas pelo sistema. Essas regras poderão ser aplicadas, por exemplo, em validações de alertas, autorização de consultas, verificação de prioridade ou tomada de decisão baseada em condições booleanas.

Na disciplina de *Prompt and Artificial Intelligence*, você utilizará os conceitos de *Large Language Models* e *Prompt Engineering* para estruturar interações simuladas com IA generativa. O projeto poderá utilizar prompts *zero-shot*, *few-shot*, saídas estruturadas e instruções bem definidas para demonstrar como um sistema pode orientar respostas em linguagem natural.

Na disciplina de modelagem linear aplicada ao aprendizado de máquina, você deverá considerar os conceitos de otimização de modelos, função de custo, erro, regularização e generalização como base para analisar a qualidade de respostas, classificações ou estimativas simples realizadas pelo sistema. O objetivo não é treinar um modelo avançado, mas compreender como a ideia de otimização contribui para melhorar decisões e reduzir erros em sistemas inteligentes.

Em *Computer Organization and Architecture*, você deverá considerar os conceitos de memórias e barramentos para refletir sobre armazenamento, acesso a dados, persistência, fluxo de informações e limitações computacionais do sistema desenvolvido.

Já em formação social e sustentabilidade, você precisará refletir sobre diversidade, racismo estrutural, inclusão e responsabilidade social no uso de sistemas inteligentes. Essa reflexão aparecerá na forma como o sistema lida com linguagem, usuários, decisões automatizadas e possíveis vieses em respostas geradas ou simuladas.

A integração permitirá que você compreenda como sistemas inteligentes não dependem apenas de código, mas também de dados bem organizados, lógica bem estruturada, prompts bem planejados, uso responsável da IA e cuidado com impactos sociais.

4 OBJETIVOS DE APRENDIZAGEM

Ao concluir essa fase, você será capaz de compreender como sistemas computacionais podem armazenar, organizar, recuperar e interpretar informações utilizando arquivos texto, arquivos JSON e estruturas lógicas. Além disso, você será capaz de aplicar conceitos de simplificação booleana para reduzir regras condicionais, estruturar prompts para orientar respostas de sistemas baseados em linguagem natural e refletir sobre os limites, riscos e responsabilidades associados ao uso de IA generativa.

Esses conhecimentos também poderão ser relacionados a situações reais, como sistemas de atendimento automatizado, organização de prontuários e registros em hospitais, análise de chamados técnicos em empresas, controle de dados em plataformas educacionais, automação de relatórios corporativos e apoio à tomada de decisão em centrais de monitoramento.

Ao longo da atividade, você também desenvolverá a capacidade de relacionar armazenamento de dados, organização computacional, otimização de modelos, lógica digital e responsabilidade social em um único protótipo funcional, compreendendo como diferentes áreas da ciência da computação se integram na construção de sistemas inteligentes aplicáveis a diferentes contextos do mundo real.

5 ATIVIDADE INTEGRADORA – NÚCLEO COGNITIVO DA AURORA SIGER (NCAS)

Nessa fase, cada equipe deverá desenvolver um protótipo chamado Núcleo Cognitivo da Aurora Siger (NCAS).

O objetivo do sistema será registrar, organizar, consultar e interpretar informações operacionais da colônia, utilizando arquivos texto, arquivos JSON, regras lógicas e prompts estruturados.

A atividade deverá utilizar os conteúdos estudados até essa fase, como:

- Manipulação de arquivos texto e JSON;
- Função `open()`;
- Modos de abertura de arquivos: `w`, `r`, `a`, `x` e `+`;

INTELIGÊNCIA ARTIFICIAL NO COMANDO

- Métodos `read()`, `readline()`, `readlines()` e `writelines()`;
- Gerenciador de contexto `with`;
- Arquivos JSON;
- Dicionários em Python;
- Teoremas de simplificação;
- Teoremas de De Morgan;
- Álgebra booleana;
- Simplificação de expressões lógicas;
- *Large Language Models*;
- Tokens;
- Pré-treinamento;
- *Next Token Prediction*;
- *Prompt Engineering*;
- *Zero-shot Prompting*;
- *Few-shot Prompting*;
- *Chain-of-Thought*;
- *Structured Outputs*;
- Otimização de modelos;
- Erro quadrático médio;
- Gradiente descendente;
- Regularização;
- Memórias e barramentos;
- Diversidade, etnia e racismo estrutural.

5.1 Organização dos dados da colônia

Cada equipe deverá definir quais informações serão registradas pelo núcleo cognitivo. Exemplos:

- Registros de manutenção: usados para armazenar ocorrências técnicas dos módulos;
- Alertas operacionais: utilizados para registrar situações de risco ou atenção;
- Solicitações da tripulação: usadas para representar pedidos feitos ao sistema;
- Histórico de respostas: usado para armazenar respostas simuladas pelo assistente;
- Status de módulos: usado para organizar informações sobre funcionamento da colônia;
- Logs de acesso: usados para registrar interações com o sistema.

Essas informações deverão ser organizadas em arquivos texto ou JSON. Além disso, a equipe deverá justificar quais dados foram armazenados em texto e quais foram armazenados em JSON.

5.2 Manipulação de arquivos em Python

- O sistema deverá permitir operações simples com arquivos.
- As equipes deverão implementar funcionalidades como:
 - Gravar registros: salvar novas informações em arquivo;
 - Ler registros: recuperar informações armazenadas;
 - Adicionar novos dados: utilizar o modo `append` quando necessário;
 - Exibir dados salvos: apresentar informações na tela;
 - Carregar dados JSON: recuperar estruturas organizadas;
 - Salvar dados JSON: armazenar dicionários ou listas de dicionários.

O objetivo é demonstrar que o sistema consegue preservar informações mesmo após sua execução ser encerrada.

5.3 Estrutura JSON do sistema

Cada equipe deverá criar pelo menos um arquivo JSON para organizar os dados do projeto. Exemplos de arquivos possíveis incluem:

- **modulos_colonia.json** – contendo dados dos módulos da Aurora Siger;
- **alertas.json** – contendo registros de alertas operacionais;
- **interacoes.json** – contendo perguntas e respostas simuladas do assistente;
- **prompts.json** – contendo modelos de prompts utilizados no sistema.

O arquivo JSON deverá apresentar:

- Estrutura válida;
- Uso adequado de chaves e valores;
- Organização clara;
- Dados coerentes com o contexto do projeto, ou seja, informações que façam sentido para o funcionamento do Núcleo Cognitivo da Aurora Siger. Por exemplo: um alerta pode conter o nome do módulo, o tipo de ocorrência, o nível de prioridade, a data do registro e uma mensagem resumida sobre o problema identificado.

5.4 Regras lógicas e simplificação booleana

O sistema deverá conter pelo menos uma regra lógica utilizada para tomada de decisão, como:

- Liberar consulta apenas se o usuário estiver autorizado e o módulo estiver ativo;
- Gerar alerta se houver falha crítica ou consumo elevado;
- Priorizar atendimento se o pedido for urgente e o setor for essencial;
- Bloquear operação se houver falha de segurança ou inconsistência nos dados.

A equipe deverá representar essa regra como uma expressão booleana e demonstrar uma simplificação utilizando os teoremas de Simplificação ou de De Morgan.

Exemplo conceitual:

- Regra original: **ALERTA = (FALHA AND CRITICO) OR (FALHA AND NOT CRITICO).**

- Regra simplificada: **ALERTA = FALHA**.

A equipe deverá explicar por que a simplificação mantém o mesmo resultado lógico.

5.5 Engenharia de prompts e simulação de IA generativa

O núcleo cognitivo deverá possuir uma funcionalidade que simule uma interação com um assistente inteligente. Além disso, a equipe deverá criar prompts estruturados para orientar respostas do sistema.

Exemplos:

- Prompt para resumir um alerta operacional;
- Prompt para classificar uma solicitação da tripulação;
- Prompt para gerar uma resposta padronizada ao centro de controle;
- Prompt para transformar um registro técnico em linguagem mais simples;
- Prompt para gerar uma saída estruturada em JSON.

A equipe deverá apresentar pelo menos:

- Um exemplo de *prompt zero-shot*;
- Um exemplo de *prompt few-shot*;
- Um exemplo de saída estruturada.

A integração real com API de IA não será obrigatória. A equipe poderá simular as respostas localmente, desde que demonstre claramente a estrutura dos prompts e a lógica de funcionamento.

5.6 Memória, armazenamento e fluxo de dados

A equipe deverá incluir uma breve explicação sobre como o sistema utiliza armazenamento e acesso aos dados.

Essa explicação deverá relacionar o projeto com os conceitos de:

- Memória;
- Armazenamento;
- Leitura e escrita de dados;

- Fluxo de informações.

O objetivo é mostrar que, mesmo quando você trabalha apenas com código Python, arquivos texto ou JSON, esses dados não existem “soltos” no computador. Eles precisam ser gravados em algum meio de armazenamento, acessados pela memória, transportados entre componentes e processados pelo sistema.

Por exemplo, quando o núcleo cognitivo salva um alerta em um arquivo JSON, esse dado passa por operações de escrita, fica armazenado em um dispositivo físico e poderá ser recuperado depois por meio de uma operação de leitura. Assim, o projeto deve evidenciar que a organização dos dados no software depende também da forma como o computador armazena, acessa e movimenta informações internamente.

5.7 Diversidade, ética e responsabilidade no uso da IA

As equipes deverão elaborar uma reflexão sobre o uso responsável de sistemas inteligentes.

A análise deverá considerar:

- Riscos de respostas enviesadas;
- Importância da diversidade no desenvolvimento de sistemas;
- Cuidado com linguagem discriminatória;
- Impactos sociais de decisões automatizadas;
- Responsabilidade humana no uso de IA generativa.

Essa reflexão deverá estar conectada ao funcionamento do núcleo cognitivo da Aurora Siger.

6 ENTREGÁVEIS

Cada equipe deverá entregar um vídeo de apresentação do projeto de no máximo 5 minutos demonstrando o funcionamento completo do projeto desenvolvido.

O vídeo deverá ser publicado no YouTube como “Não listado” e o link deverá ser entregue juntamente com uma pasta compactada (.zip) contendo todos os arquivos do projeto.

Durante o vídeo, a equipe deverá apresentar:

- Descrição do núcleo cognitivo da Aurora Siger;
- Explicação dos dados armazenados pelo sistema;
- Demonstração da leitura e gravação de arquivos;
- Explicação do arquivo JSON utilizado;
- Apresentação da regra lógica e sua simplificação;
- Explicação dos prompts criados;
- Demonstração da interação simulada com IA generativa;
- Análise simples de otimização ou melhoria de resposta;
- Reflexão sobre diversidade, ética e responsabilidade;
- Demonstração prática do funcionamento do sistema em Python.

6.1 Requisitos do sistema desenvolvido

O sistema entregue pela equipe deverá conter:

- Código em Python;
- Manipulação de arquivos texto;
- Leitura e escrita de arquivo JSON;
- Pelo menos uma regra lógica implementada;
- Pelo menos uma simplificação booleana explicada;
- Prompts estruturados;
- Simulação de resposta inteligente;
- Organização e comentários no código.

Além disso, o sistema deverá apresentar:

- Menu simples de navegação no terminal;
- Dados organizados de forma clara;
- Mensagens de entrada e saída claras;

- Exemplos de execução das funcionalidades implementadas.

As funcionalidades implementadas deverão permitir:

- Cadastrar registros da colônia;
- Consultar registros salvos;
- Carregar dados em JSON;
- Executar uma validação lógica;
- Exibir prompts estruturados;
- Simular uma resposta do assistente inteligente.

Exemplo de funcionalidade esperada: ao selecionar a opção “Analisar alerta operacional”, o sistema poderá carregar um alerta salvo em JSON, verificar se ele é crítico utilizando uma regra booleana simplificada e exibir um prompt estruturado para gerar uma resposta padronizada ao centro de controle.

Durante a apresentação do vídeo, a equipe deverá demonstrar o funcionamento real do sistema.

6.2 Arquivos obrigatórios na entrega (.zip)

A pasta compactada **.zip** deverá conter, obrigatoriamente, os seguintes arquivos, com os respectivos formatos e nomes padronizados:

- **codigo_fonte.py** - arquivo principal do sistema, contendo o código-fonte em Python;
- **dados_colonia.json** - arquivo JSON com os dados estruturados utilizados pelo sistema;
- **registros_colonia.txt** - arquivo texto com registros, logs ou informações salvas pelo sistema;
- **regras_logicas.pdf** - arquivo contendo a expressão booleana utilizada, sua simplificação e explicação;
- **prompts_utilizados.pdf** - arquivo contendo os prompts criados pela equipe e a explicação de cada um;

- **link_video.txt** - arquivo de texto contendo o link do vídeo da apresentação publicado no YouTube como “Não listado”.

Importante:

- Todos os arquivos devem estar organizados de forma clara dentro da pasta **.zip**;
- Caso o projeto possua mais de um arquivo Python, a equipe deverá manter os nomes organizados e intuitivos;
- O arquivo principal do sistema deverá estar identificado de forma clara para facilitar a correção.

6.3 Regras técnicas

O código deverá ser executado sem necessidade de bibliotecas avançadas ou *frameworks* externos, utilizando apenas recursos compatíveis com os conteúdos estudados até essa fase.

A integração real com APIs de IA é opcional. Caso a equipe utilize alguma API, deverá explicar claramente como ela foi utilizada. Caso não utilize, poderá simular o comportamento do assistente inteligente com prompts e respostas predefinidas.

7 CRITÉRIOS DE AVALIAÇÃO

Critério	Descrição	Pontuação
Integração das disciplinas	Capacidade de integrar os conteúdos da fase ao projeto	2,0
Manipulação de arquivos e JSON	Uso correto de arquivos texto, JSON, leitura e escrita em Python	2,0
Lógica booleana e simplificação	Aplicação correta de regras lógicas, simplificação e De Morgan	1,5
<i>Prompt Engineering</i> e LLMs	Qualidade dos prompts, simulação de IA e clareza das respostas estruturadas	2,0
Organização computacional	Estrutura, funcionamento, comentários e clareza do sistema	1,5
Ética, diversidade e responsabilidade	Relação entre IA, diversidade, vieses e uso responsável da tecnologia	1,0

Total		10,0
-------	--	------

Tabela 1 – Critérios de avaliação
Fonte: Elaborada pelo autor (2026)

EMANIP

GLOSSÁRIO

Arquivo texto	Arquivo utilizado para armazenar informações em formato textual, podendo ser lido e escrito por programas em Python.
JSON	Formato estruturado de dados baseado em chaves e valores, utilizado para armazenar e transportar informações entre sistemas.
Dicionário	Estrutura de dados em Python formada por pares de chave e valor, muito utilizada para manipular dados semelhantes ao formato JSON.
Prompt	Instrução enviada a um modelo de linguagem para orientar a geração de uma resposta.
<i>Prompt Engineering</i>	Prática de projetar e organizar prompts para obter respostas mais úteis, claras e alinhadas ao objetivo desejado.
LLM	<i>Large Language Model</i> . Modelo de linguagem treinado em grandes volumes de texto para prever e gerar sequências de tokens.
Token	Unidade de texto processada por modelos de linguagem, podendo representar palavra, parte de palavra ou símbolo.
<i>Zero-shot</i>	Técnica em que o modelo recebe apenas a instrução da tarefa, sem exemplos.
<i>Few-shot</i>	Técnica em que o modelo recebe alguns exemplos antes de executar uma nova tarefa semelhante.
<i>Structured Output</i>	Resposta organizada em formato estruturado, como JSON, tabela ou lista padronizada.
Álgebra booleana	Área da matemática que trabalha com valores lógicos 0 e 1 e operações como AND, OR e NOT.
De Morgan	Conjunto de teoremas utilizados para transformar expressões booleanas envolvendo negação, AND e OR.

INTELIGÊNCIA ARTIFICIAL NO COMANDO

Simplificação booleana	Processo de reduzir uma expressão lógica mantendo o mesmo resultado.
Otimização de modelos	Processo de ajustar parâmetros ou estratégias para reduzir erros e melhorar o desempenho de um modelo.
Regularização	Técnica utilizada para controlar a complexidade de modelos e melhorar sua capacidade de generalização.
Memória	Componente responsável por armazenar informações em sistemas computacionais.
Viés algorítmico	Tendência de um sistema produzir resultados injustos ou distorcidos por causa dos dados, regras ou decisões usadas em seu desenvolvimento.